Alunos: Ozaeltton Matheus da Silva Medeiros (20231038060022) e Kauê Adriano dos Santos Dantas (20241038060007)

Mudanças na justificativa e na problemática

Ao aprofundarmos na análise do conjunto de dados, percebemos que a pergunta original não é sustentada, pois não possui colunas necessárias para se analisar os fatores que são decisivos para um jogo ter sucesso no Ocidente e no Oriente. Afinal, não temos campos de orçamento, número de jogadores, etc. Além de que todos os jogos listados são bem sucedidos quanto ao número de vendas.

Após reformulação

A divisão permanece: Ocidente (América do Norte + Europa) e Oriente (Japão). Frisamos novamente que a coluna “Other Sales” não é considerada, tendo em vista que o número une as vendas de diferentes regiões do mundo, sem divisão.

Com isso, a problemática central passa a ser “como o perfil dos jogos mais vendidos, em termos de gênero, plataforma e editora (publisher), difere entre o Ocidente e o Oriente?”

Conjunto de dados

O Video Game Sales (Kaggle, autor gregorut) contém 16.598 registros e 11 colunas, com vendas em milhões de unidades por região (NA_Sales, EU_Sales, JP_Sales, Other_Sales, Global_Sales) e as variáveis Rank, Name, Platform, Year, Genre e Publisher. Cada linha representa um jogo em uma plataforma específica.

Procedimentos e pré-processamento justificados

1. Primeiro fizemos uma inspeção do conjunto de dados, para verificar campos vazios e duplicados.
2. Removemos colunas desnecessárias para análise: **Rank, Global_Sales, Other_Sales e Name.**

- Rank é calculado a partir de **Global_Sales**, e **Global_Sales** inclui as vendas das regiões usadas para construir a variável-alvo. Mantê-las causaria vazamento de informação.
- **Other_Sales** está fora do recorte Ocidente × Oriente.
- **Name** é apenas um identificador.

3. Decidimos manter a coluna **Year**. Na primeira versão do código, essa coluna era removida. Decidimos mantê-la, pois a EDA mostrou que plataformas e gêneros variam fortemente com a época de lançamento. Sem o ano, a diferença entre Ocidente e Oriente se confundiria com a diferença entre gerações de consoles.
4. Ao analisarmos os valores nulos, faltantes e zerados, concluímos que **Publisher** possui 58 valores nulos, e **Year** também apresenta valores ausentes. Os nulos de Publisher foram preenchidos com "Unknown", preservando as linhas. Os de **Year**, com a mediana, por ser menos sensível a valores extremos. Ambas as imputações ficam dentro de pipelines ajustadas apenas no treino.
5. Quanto aos valores zero nas vendas, decidimos manter os zeros, pois indicam que o jogo vendeu menos de 10 mil cópias na região, no arredondamento do dataset. É uma informação relevante sobre o mercado, principalmente para o Japão, e é a partir dela que se calcula a participação de cada região nas vendas do jogo.
6. Removemos jogos sem vendas nas regiões de interesse. Portanto, foram removidos os jogos sem vendas na América do Norte, na Europa e no Japão (2 registros, que só tinham vendas em **Other_Sales**). Sem vendas nessas regiões, não é possível calcular a participação de cada uma e, portanto, não há como atribuir uma classe ao jogo.
7. Construímos a variável-alvo. Esta etapa mudou em relação à versão anterior, em que o alvo eram duas variáveis numéricas (**West_Sales** e **East_Sales**). Como as etapas seguintes do projeto utilizam algoritmos de classificação e comparam os grupos da clusterização com uma coluna-alvo, passamos a usar uma variável categórica, **Mercado**.
8. Primeiro, agrupamos as regiões: **Ocidente = América do Norte + Europa** e **Oriente = Japão**. Comparar diretamente as vendas dos dois grupos favorece o Ocidente por construção, pois duas regiões somadas são comparadas a uma só. Por isso, a classe é definida pela participação do Japão nas vendas de cada jogo:

Participacao_Oriente = JP_Sales / (NA_Sales + EU_Sales + JP_Sales)

Classe	Critério	Significado
Ocidente	Participação do Japão < 10%	Vendas concentradas no Ocidente
Equilibrado	Entre 10% e 50%	Vendas distribuídas entre as regiões
Oriente	Participação do Japão > 50%	Vendas concentradas no Japão

Assim, a classe mede se o jogo vende de forma relativamente melhor em uma região, independentemente do tamanho de cada mercado. Após a criação da coluna Mercado, as colunas de vendas e a participação foram retiradas das características, evitando vazamento.

9. Fizemos a codificação das variáveis categóricas. **Platform, Genre e Publisher** foram transformadas por one-hot encoding. Categorias com menos de 10 jogos (comuns em Publisher) foram agrupadas em uma categoria “infrequent”, evitando centenas de colunas quase vazias.
10. Fizemos a padronização de variáveis numéricas. **Year** foi padronizada (média 0 e desvio 1) para ter escala comparável às demais colunas.
11. Dividimos treino/teste. Dividimos em **80% para treino** e **20% para teste**. A divisão é estratificada pela variável-alvo, mantendo a proporção das três classes nos dois conjuntos, o que é importante porque as classes são desbalanceadas. As transformações foram ajustadas somente no treino e apenas aplicadas no teste, evitando vazamento de informação.

Resultado

Após a remoção de 2 jogos, o conjunto tem 16.596 registros, divididos em 13.276 para treino e 3.320 para teste, ambos com 176 colunas numéricas e sem valores nulos.

```
--------------------Distribuição da variável-alvo (Mercado)--------------------
Mercado
Ocidente       11500
Oriente         3853
Equilibrado     1243
Name: count, dtype: int64
Mercado
Ocidente       0.693
Oriente        0.232
Equilibrado    0.075
Name: proportion, dtype: float64
--------------------Resultado do pré-processamento--------------------
Treino: (13276, 176) | Teste: (3320, 176)
Nº de colunas após codificação: 176
```

A variável-alvo **Mercado** apresentou a seguinte distribuição:

A maior parte dos jogos concentra suas vendas no Ocidente, mas quase um quarto vende majoritariamente no Japão, o que confirma a existência de um perfil de jogos do Oriente a ser analisado.

Limitações

- O Japão representa o Oriente de forma simplificada, e o dataset só inclui jogos com mais de 100 mil cópias vendidas.
- Os limiares de 10% e 50% que definem as classes são uma decisão do grupo. Outros valores alterariam a distribuição das classes e, possivelmente, os resultados.
- As classes são desbalanceadas, e a classe Equilibrado é a menor e a mais difícil de prever, pois fica numa zona intermediária entre as outras duas.
- Um mesmo jogo lançado em várias plataformas ocupa várias linhas e pode aparecer no treino e no teste ao mesmo tempo, o que pode inflar o desempenho de um modelo futuro.
- Com apenas quatro características (Platform, Genre, Publisher e Year), o poder preditivo dos modelos tende a ser limitado.

Código

```
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split

# Parâmetros da variável-alvo
# Participação do Japão nas vendas (NA + EU + JP) de cada jogo:
# abaixo de LIMIAR_OCIDENTE -> "Ocidente"
# acima de LIMIAR_ORIENTE   -> "Oriente"
# entre os dois             -> "Equilibrado"
LIMIAR_OCIDENTE = 0.10
LIMIAR_ORIENTE = 0.50

data = pd.read_csv('./vgsales.csv')

print("-" * 20 + "Primeiras linhas do dataset" + "-" * 20)
print(data.head())

print("-" * 20 + "Últimas linhas do dataset" + "-" * 20)
print(data.tail())

print("-" * 20 + "Formato do dataset" + "-" * 20)
print(data.shape)  # esperado: cerca de 16.598 linhas e 11 colunas

print("-" * 20 + "Tipos das colunas" + "-" * 20)
print(data.dtypes)

# Valores nulos por coluna (o pandas já lê "N/A" do CSV como nulo).
# Publisher e Year possuem nulos; as colunas de vendas não.
print("-" * 20 + "Valores nulos por coluna" + "-" * 20)
print(data.isnull().sum())

# Linhas duplicadas
print("-" * 20 + "Linhas duplicadas" + "-" * 20)
print(data.duplicated().sum())

# Os valores 0 nas colunas de vendas têm significado para a análise (o
jogo
```

```
# vendeu menos de 10 mil cópias na região, no arredondamento do dataset),
# portanto não são apagados nem substituídos.

# Exclusão de colunas desnecessárias para a análise:
# Rank: é calculado a partir de Global_Sales, que contém as vendas das
# regiões usadas para criar o alvo (vazamento de informação).
# Global_Sales: soma das regiões, incluindo as que usamos para o alvo.
# Other_Sales: fora do recorte Ocidente x Oriente.
# Name: identificador (milhares de valores únicos), sem poder de generalização.
# A coluna Year é MANTIDA: plataformas e gêneros mudam muito com a época,
# e o ano ajuda a separar o efeito temporal do efeito cultural.
data = data.drop(columns=['Rank', 'Name', 'Other_Sales', 'Global_Sales'])

# Ocidente = América do Norte + Europa; Oriente = Japão.
data["West_Sales"] = data["NA_Sales"] + data["EU_Sales"]
data["East_Sales"] = data["JP_Sales"]
total_regioes = data["West_Sales"] + data["East_Sales"]

# Jogos sem vendas nas três regiões (vendas apenas em Other_Sales) não têm
# como receber uma classe e são removidos.
sem_vendas = total_regioes == 0
print("-" * 20 + "Jogos removidos (sem vendas em NA, EU e JP)" + "-" * 20)
print(int(sem_vendas.sum()))
data = data[~sem_vendas].copy()
total_regioes = total_regioes[~sem_vendas]

# Comparar West_Sales com East_Sales diretamente favoreceria o Ocidente por
# construção (duas regiões somadas contra uma). Por isso o alvo usa a
# participação do Japão nas vendas de cada jogo: mede se o jogo performa de
# forma relativamente melhor no Oriente, independentemente do tamanho do mercado.
data["Participacao_Oriente"] = data["East_Sales"] / total_regioes

# Construção da variável-alvo (Mercado) a partir da participação do Japão nas vendas.
```

```
# Indica qual mercado o jogo vende mais: Ocidente, Oriente ou
Equilibrado.
data["Mercado"] = np.select(
   [
       data["Participacao_Oriente"] < LIMIAR_OCIDENTE,
       data["Participacao_Oriente"] > LIMIAR_ORIENTE,
   ],
   ["Ocidente", "Oriente"],
   default="Equilibrado",
)

print("-" * 20 + "Distribuição da variável-alvo (Mercado)" + "-" * 20)
print(data["Mercado"].value_counts())
print(data["Mercado"].value_counts(normalize=True).round(3))
# Classes desbalanceadas: nas próximas etapas usar validação cruzada
# estratificada e métricas como F1 macro / acurácia balanceada.

# As colunas de vendas e derivadas saem das características, evitando
vazamento de informação.
colunas_vendas = ["NA_Sales", "EU_Sales", "JP_Sales",
                  "West_Sales", "East_Sales", "Participacao_Oriente"]

y = data["Mercado"]
X = data.drop(columns=colunas_vendas + ["Mercado"])

colunas_categoricas = ['Platform', 'Genre', 'Publisher']
colunas_numericas = ['Year']

# Pré-processamento abaixo
# Categóricas: nulos viram "Unknown"; categorias raras (poucos jogos,
comum em
# Publisher) são agrupadas em uma categoria "infrequente" para evitar
centenas
# de colunas quase vazias.
pipeline_categorico = Pipeline([
                     ('imputer',       SimpleImputer(strategy='constant',
fill_value='Unknown')),
        ('onehot',   OneHotEncoder(handle_unknown='infrequent_if_exist',
min_frequency=10)),
])

# Numéricas: nulos viram a mediana; depois padronização.
pipeline_numerico = Pipeline([
```

```
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()),
])

# Este "preprocessor" pode ser usado dentro de um Pipeline com o classificador
# nas próximas etapas, para ser reajustado a cada dobra da validação cruzada.
preprocessor = ColumnTransformer(
    transformers=[
        ('cat', pipeline_categorico, colunas_categoricas),
        ('num', pipeline_numerico, colunas_numericas),
    ]
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

X_train_proc = preprocessor.fit_transform(X_train)   # aprende só no treino
X_test_proc = preprocessor.transform(X_test)         # apenas aplica no teste

print("-" * 20 + "Resultado do pré-processamento" + "-" * 20)
print("Treino:", X_train_proc.shape, "| Teste:", X_test_proc.shape)
print("N° de colunas após codificação:", len(preprocessor.get_feature_names_out()))
```